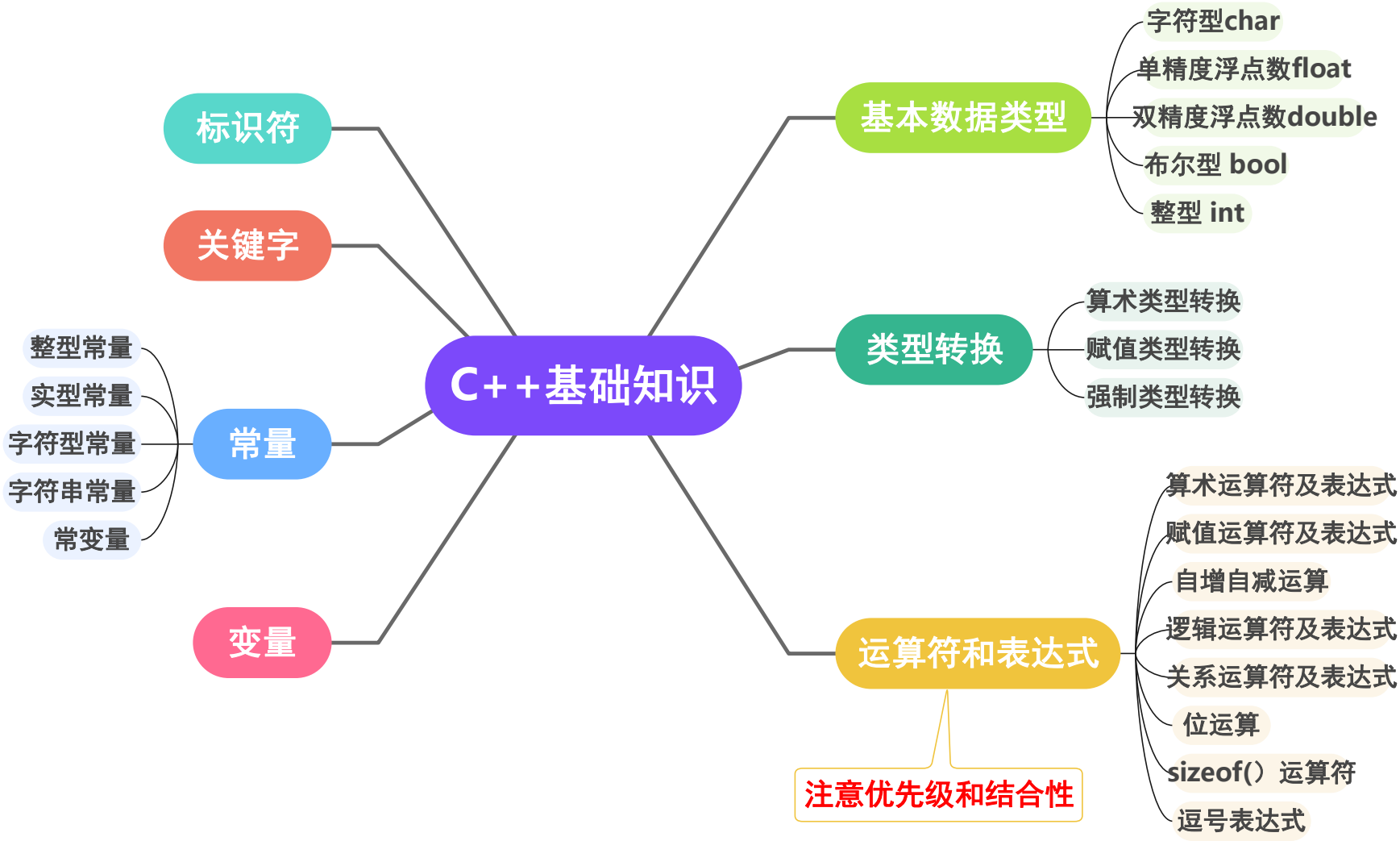
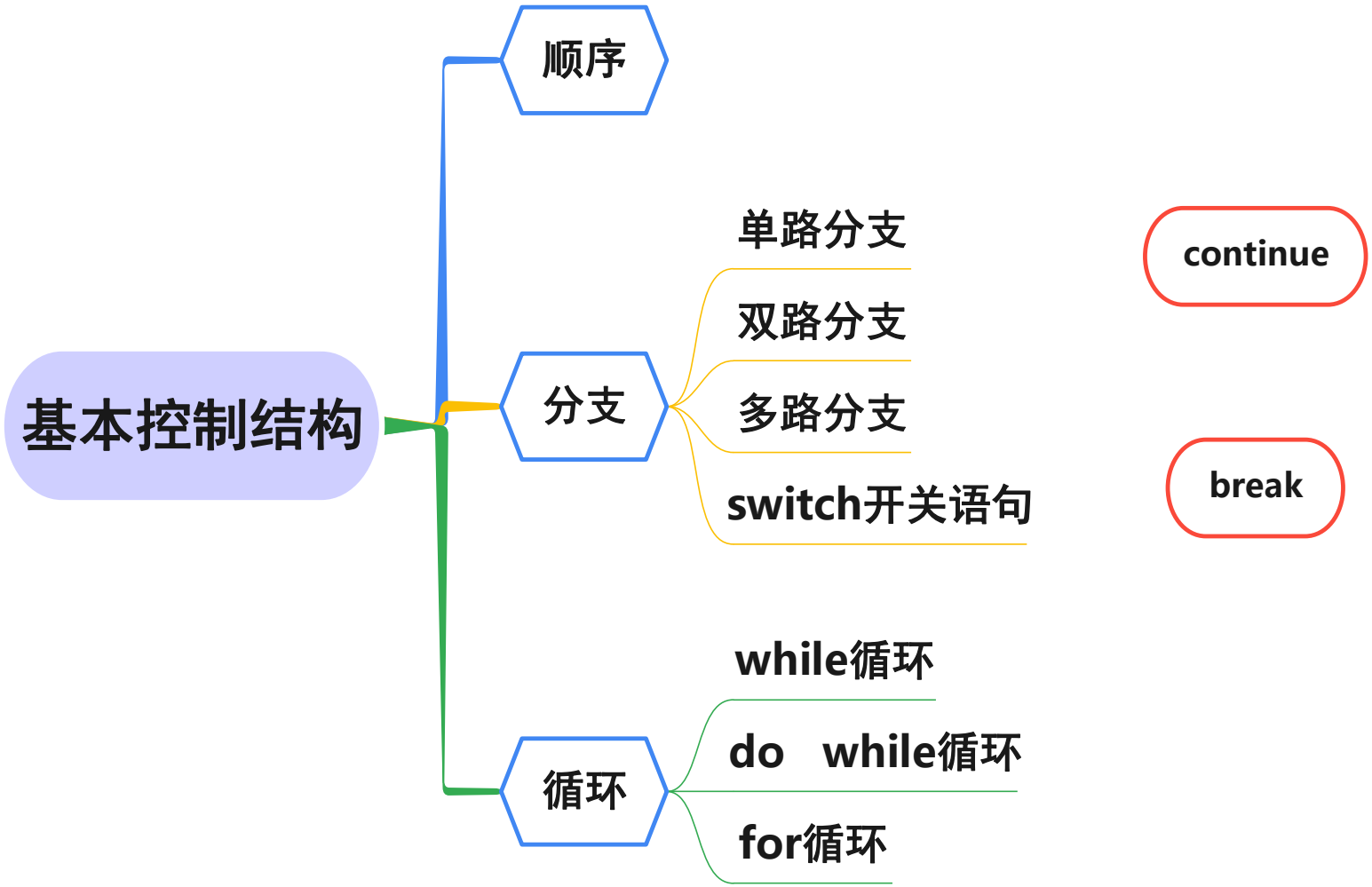
程序设计与算法语言II

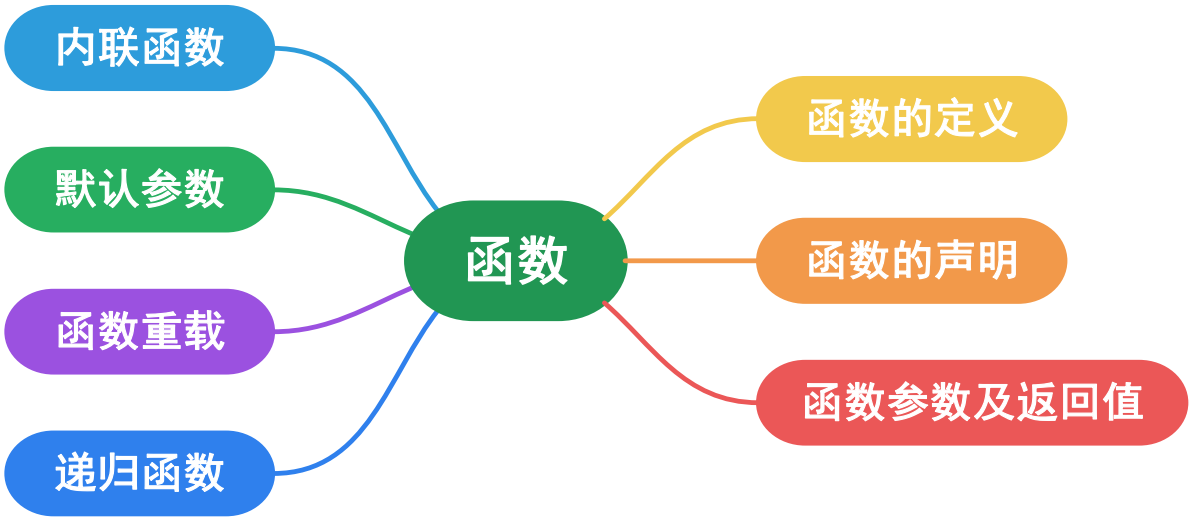
上学期内容回顾


C++基础知识
标识符
关键字
常量
整型常量
实型常量
字符型常量
字符串常量
常变量
变量
基本数据类型
字符型char
单精度浮点数float
双精度浮点数double
布尔型 bool
整型 int
类型转换
算术类型转换
赋值类型转换
强制类型转换
运算符和表达式
算术运算符及表达式
赋值运算符及表达式
自增自减运算
逻辑运算符及表达式
关系运算符及表达式
位运算
sizeof()运算符
逗号表达式
注意优先级和结合性

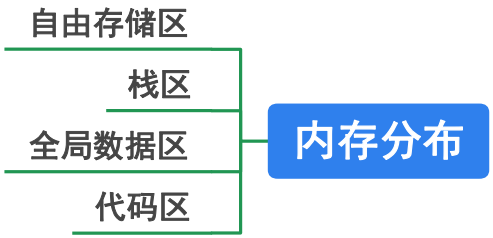
上学期内容回顾


基本控制结构
顺序
分支
单路分支
双路分支
多路分支
switch开关语句
循环
while循环
do while循环
for循环
continue
break

上学期内容回顾



全局变量　局部变量　局部静态变量

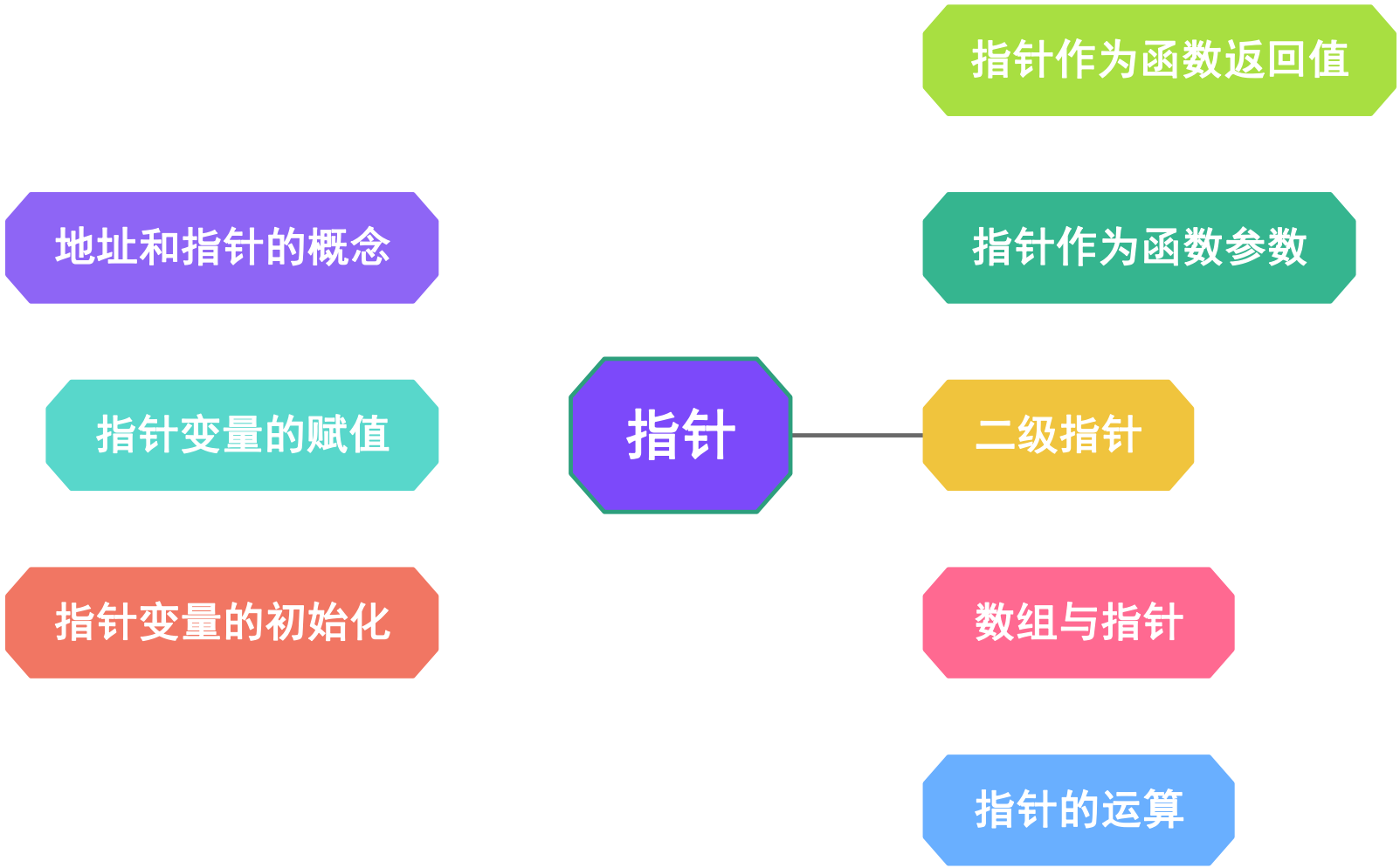








上学期内容回顾


指针作为函数返回值
地址和指针的概念
指针作为函数参数
指针变量的赋值
指针
二级指针
指针变量的初始化
数组与指针
指针的运算

第一讲 结构、类与对象

- struct
- union
- class && object

结构（struct）

结构概念的引入

如何对表格信息进行存储？

学号	姓名	性别	年龄	成绩	住址
N001	张三	男	18	89	四牌楼2号
N002	李四	男	17	79	北京东路1号
N003	王五	女	17	90	北京西路1号
N004	赵六	男	18	82	上海路1号

数组？

按照个人信息一组一组的存储

结构体定义

将不同种类型的数据有序地组合在一起，构造出一个新的数据类型，这种形式称为结构体（struct）。

结构体是多种类型组合的数据类型。

```
struct 结构类型名 {
    类型名  变量1;
    类型名  变量2;
        …
    类型名  变量n;
};
```

可以不同类型

分号不可少

```
struct  student
  {  char id[8];
     char name[20]，sex;
     int  score;
     char addr[30];
   };
```

结构体定义

- 结构体成员的数据类型包括：

  基本数据类型及其数组和已定义过的某种数据类型及其数组。

- 在一个结构类型中各个成员的名字不能重复。

- 定义一个结构类型是一条完整的说明语句，所有成员用一对花括号括起来，最后用分号结束。

- 不能指定成员为auto、register、extern，可以为static。

结构变量

1）先定义结构体类型再定义变量名

```
struct   student
  {     int  num;
        char  name[20];
        char  sex;
        int  age;
        float  score;
        string  addr;
};
```

结构体类型定义

结构体类型只是一种数据类型，不占内存空间，只有定义结构体类型变量时才开辟内存空间。

```
student   student1,  student2;
```

结构变量定义

结构体类型名　变量1　变量2

结构变量

2）在定义类型的同时定义变量

```
struct  student
  {   int  num;
      char  name[20];
      char  sex;
      int  age;
      float  score;
      char  addr[30];
} student1, student2;
```

紧接着定义变量

```
struct   结构体名
      {
           成员列表
      } 变量名列表;
```

结构变量

- 结构体类型的变量在内存依照其成员的顺序顺序排列，所占内存空间的大小是其全体成员所占空间的总和。
- 在编译时，仅对变量分配空间，不对类型分配空间。

- 对结构体中各个成员可以单独引用、赋值，其作用与变量等同。

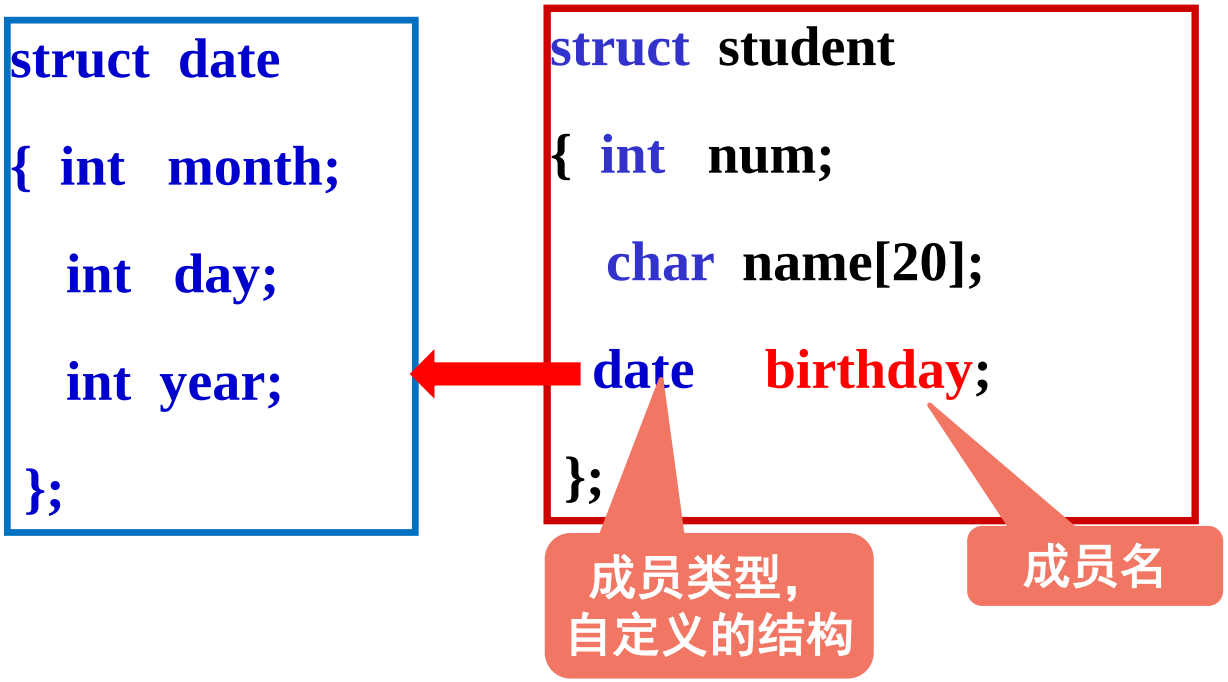
点操作符

引用格式：变量名 . 成员名

student1 . num

结构变量

- 结构体的成员可以是另一个结构体类型。



- 结构体内的成员名可以与程序中的变量名相同，二者分占不同的内存单元，互不干扰。

结构变量的初始化


```
struct  student
  {   int  num;
      char  name[20];
      char  sex;
      char  addr[30];
   };
int main( )
{
      student  student1={1001, “Li Lin”, ‘M’, “Beijing Road”};
      cout<<student1.name<<endl;
      return 0;
}
```

对结构变量初始化，各个成员一一赋值

输出：　LiLin

结构变量的引用

- 不能直接对结构体变量整体赋值或输出，只能分别对各个成员引用。

必须用成员名引用

错误 `cin>>student1;`

```
cin>>student1.num;
student1.num=100;
```

- 可以将一个结构体变量整体赋给另外一个相同类型的结构体变量。

```
student2=student1;
```

- 嵌套的结构体变量必须逐层引用。

```
student1.birthday.day=25;
```

- 结构体变量中的成员可以同一般变量一样进行运算。

```
student1.birthday.day++;
student1.score+=60;
```

结构示例1

结构体数组

结构体数组中的每个元素都是一个结构体类型的变量，其中包括该类型的各个成员。数组各元素在内存中连续存放。

结构体数组的定义

```
struct  student
  {   int  num;
      char  name[20];
      char  sex;
      int  age;
      float  score;
      char  addr[30];
} ;
struct  student  stu[30];
```

```
struct  student
  {   int  num;
      char  name[20];
      char  sex;
      int  age;
      float  score;
      char  addr[30];
} stu[30];
```

直接定义

结构体数组的初始化

```
struct   student
  {    int  num;
       char  name[20];
       char  sex;
} stu[ ]={ {1011, "Li Lin",'M'}, {1012,"Wang Lan",'F'},
            {1013,"Liu Fang",'F'}};
```

```
struct   student
  {    int  num;
       char  name[20];
       char  sex;
} stu[3]={1011,"Li Lin",'M‘,1012,"Wang Lan",'F,
            1013,"Liu Fang",'F'};
```

根据下面的定义，能打印出大写字母M的语句是：

```
struct  person
{
    char  name[9];
    int  age;
};
struct  person class[10]={  “Jone”,17, “Paul”,19,
                            “Mary”,18, “Adam”,16
                          };
```

A) cout<<class[3].name<<endl;　　输出：Adam

B) cout<<class[1].name[1]<<endl;　　输出：a

C) cout<<class[2].name[1]<<endl;　　输出：a

D) cout<<class[2].name[0]<<endl;　　输出：M

结构类型作为函数的参数与返回值

- 结构类型可作为函数的形参或返回值。

  void print (Student stdu);

- 实参变量将赋值给形参(将复制所有成员)，然后开始执行函数。
- 当函数返回一个结构变量时，通过一次赋值来得到结果。
- 性能很低，很少用。
- 结构的指针或引用作为形参或返回值更好。

void print (Student *stdu)　　void print (Student &stdu)

通过指针访问结构成员


```
#include<iostream>
#include<cstring>
using namespace std;
struct Student
{
        char name[20];
        char id[10];
        int age;
        float score;
        char address[30];
};
```

通过指针访问结构成员

结构变量

结构指针变量

结构示例2

```
int main(){
    Student st1,*prst;
    prst=&st1;
    cout<<"请输入学生的姓名： "<<endl;
    cin.getline(st1.name,20);
    cout<<"请输入学号、年龄和入学成绩： "<<endl;
    cin>>st1.id>>st1.age>>st1.score;
    cin.get();      //吸收掉换行回车符
    cout<<"请输入家庭住址： "<<endl;
    cin.getline(st1.address,30);
    cout<<“姓名： ”<<prst->name<<‘\n<<“学号： "<<prst->id<<'\n';
    cout<<"年龄"<<'\t'<<prst->age<<'\n';
    cout<<"入学成绩"<<'\t'<<prst->score<<'\n';
    cout<<"家庭住址"<<'\t'<<prst->address<<endl;
    return 0;}
```

指针变量后用箭头操作符（->）访问数据成员

结构体类型中的静态成员

static修饰的成员称为静态成员。

当把结构体类型中的某一个成员的存储类型定义为静态时，表示在这种结构类型的所有变量中，编译程序为这个成员只分配一个存储空间，即这种结构体类型的所有变量共同使用这个成员的存储空间。

结构体类型中的静态成员

结构体静态成员的特点如下：

- 静态成员的值为该结构类型的所有变量所共享，通过一个结构变量能修改静态成员值，其它结构变量也能看到这个变化。
- 在运行时期静态成员的值将存储在静态存储区中，其生存期是从程序开始执行到程序结束退出为止。因此即便没有定义该结构类型的任何变量，该结构类型的静态成员的值也存在而且可访问。

结构体类型中的静态成员

※ 必须在结构体之外为静态成员分配空间并设置初值。

格式：

<数据类型> <结构类型>::<静态成员名> [= <初值>];

其中，::是作用域运算符，当置初值时，缺省值为0。

```
struct Employee{
        char num[5],name[20];
        char sex;
        Date birthday;
        float salary;
        static float lowerestSalary;//静态成员的说明
};
float Employee::lowerestSalary = 800;//静态成员定义及初始化
```

这时，未定义结构体变量，但已将静态成员的空间安排好。

数据类型　结构体类型　静态结果成员

若有定义：Employee e1,e2; 则变量e1,e2的lowerestSalary成员占用同一存储空间（静态区）。

共用体（union）

共用体（联合）定义及变量

C++语言中，允许不同的数据类型使用同一存储区域，即同一存储区域由不同类型的变量共同表示。这种数据类型就是共用体。

```
union 共用体名
{ 成员表列;
}变量表列;
```

```
union data
{
    int i;
   char ch;
   float f;
};
```

```
union data a, b, c;
```

这几个成员在共用体变量中存放在同一地址，相互覆盖，其长度为最长的成员的长度。

共同体变量的引用

※ 不能整体引用共同体变量，只能引用变量中的成员

a.i　表示为整型

a.ch　表示为字符型

a.f　表示为符点型

共同体由若干成员组成，这些成员共享同一块存储空间，在某一时刻只有一个成员在起作用；

共同体

- 共用体变量中的成员是最后一次放入的成员；
- 共同体变量的大小就是其中最大成员的大小；
- 共用体变量不能在定义时赋初值；
- 共用体变量不能作为函数的参数或函数值，但可使用指向共用体的指针变量；
- 共用体可以作为结构的成员，结构体也可以作为共用体的成员。

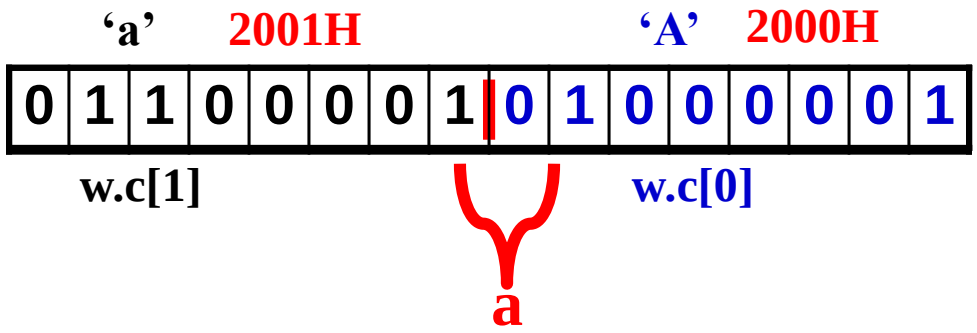
共同体（联合）示例1

低字节低地址

高字节高地址

```
union    un
{
      short a;
      char  c[2];
} ;
un w;
w.c[0]='A';   w.c[1]='a';
cout<<oct<<w.a<<endl;
```

输出：060501



联合示例1

共同体（联合）示例2


```
union HackFloat {
        float floatValue;
        unsigned char byteValue[4];
};
union HackDouble{
        double doubleValue;
        unsigned char byteValue[8];
};
void hackIntoFloat()
{
   union HackFloat floatData;
   float Data.floatValue= 176.0625; //对应  43 30 10 00
   for (int i=3;i>=0;i--)
   {      cout<< “byte_value of floatVar is” ;
       cout<<hex<<(int)floatData.byteValue[i]<<endl;
   }
}
```

共同体（联合）示例2

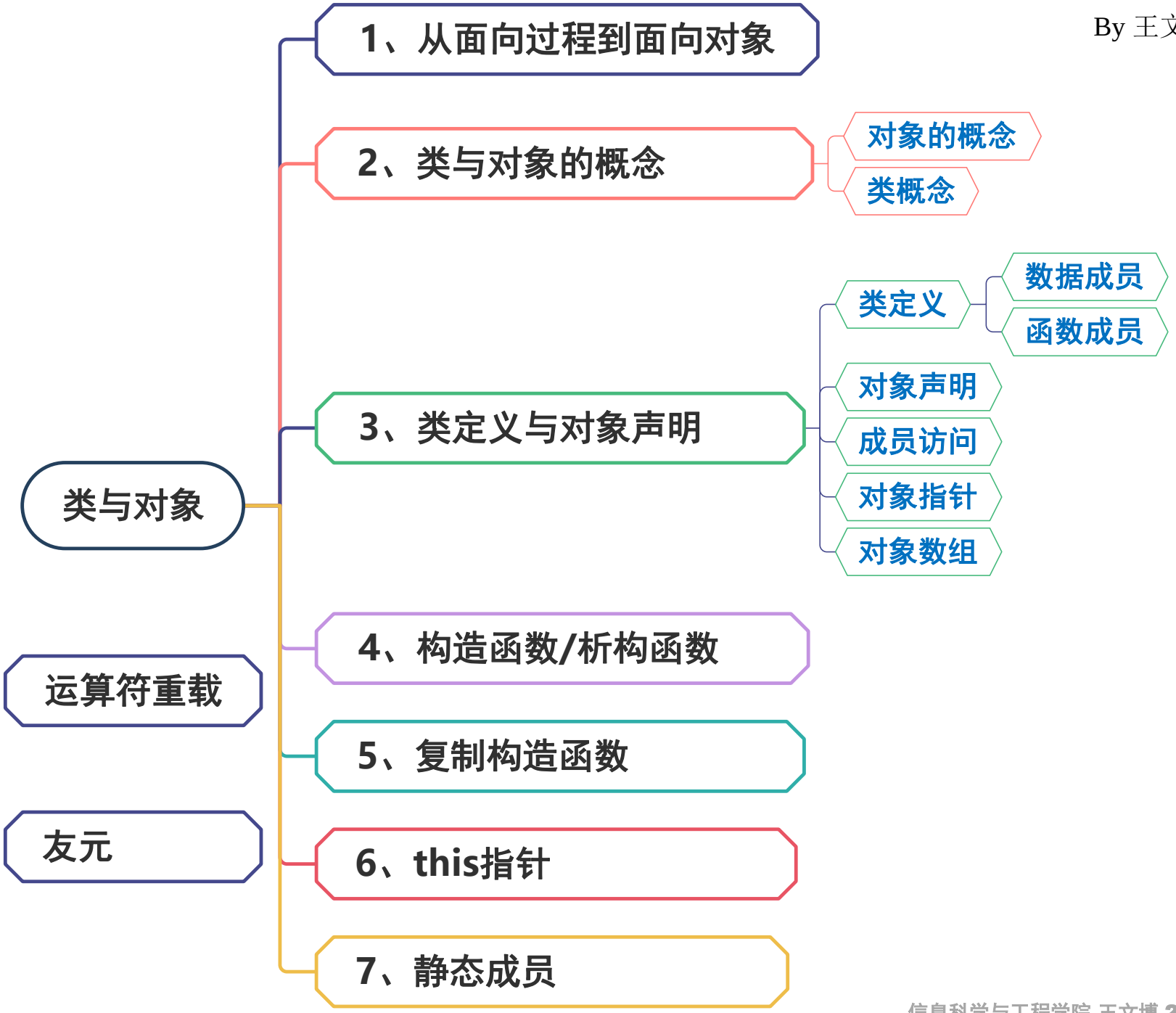

```
void hackIntoDouble()
{
  union HackDouble DoubleData;
  double Data.doubleValue= 176.0625;
  //对应  40 66 02 00 00 00 00 00
  for (int i=7;i>=0;i--)
   {
    cout<<“byte_value of doubleVar is”;
    cout<<hex<<(int)DoubleData.byteValue[i]<<endl;
     //强制转换为int后以16进制格式输出
   }
}
```

```
int main()
{
        hackIntoFloat();
         hackIntoDouble();
            return 0;
}
```

联合示例2

类与对象

By 王文博


类与对象
1、从面向过程到面向对象
2、类与对象的概念
对象的概念
类概念
3、类定义与对象声明
类定义
数据成员
函数成员
对象声明
成员访问
对象指针
对象数组
4、构造函数/析构函数
5、复制构造函数
6、this指针
7、静态成员
运算符重载
友元

1、从面向过程到面向对象

面向过程的结构化设计方法

注重细节的实现过程，将数据与函数分开。

- 特点：
  - 自顶向下，逐步求精——功能分解
- 形式：
  - 主模块+若干个子模块（main()+子函数）

各模块间的关系尽可能简单，功能上相对独立；每一模块内部均是由顺序、选择和循环三种基本结构组成。

1、从面向过程到面向对象

面向过程的结构化设计方法

优点

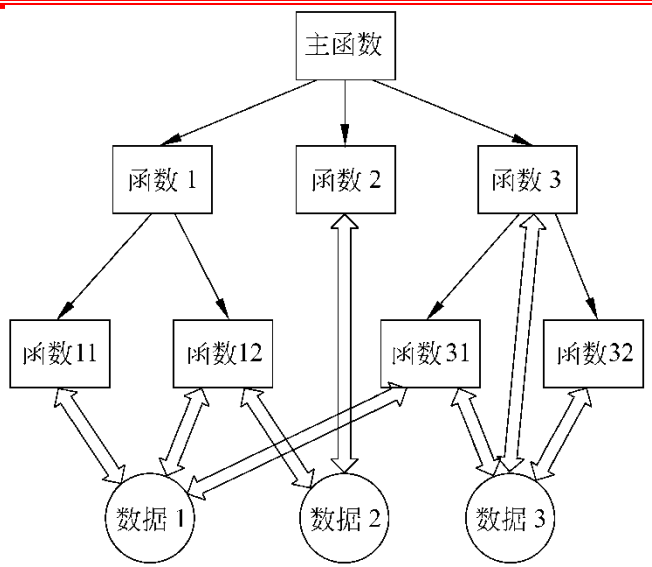
有效地将一个较复杂的程序系统设计任务分解成许多易于控制和处理的子任务，便于开发和维护。

缺点

可重用性差

数据安全性差

难以开发图形界面的应用



数据和处理数据的过程分离为相互独立实体。当数据结构改变时，所有相关处理过程都要进行相应修改。

所有的数据都是公用的，一个函数可以使用任何一组数据，而一组数据又能被多个函数所使用

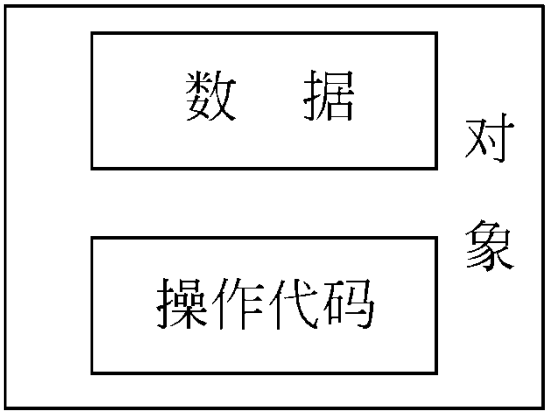
1、从面向过程到面向对象

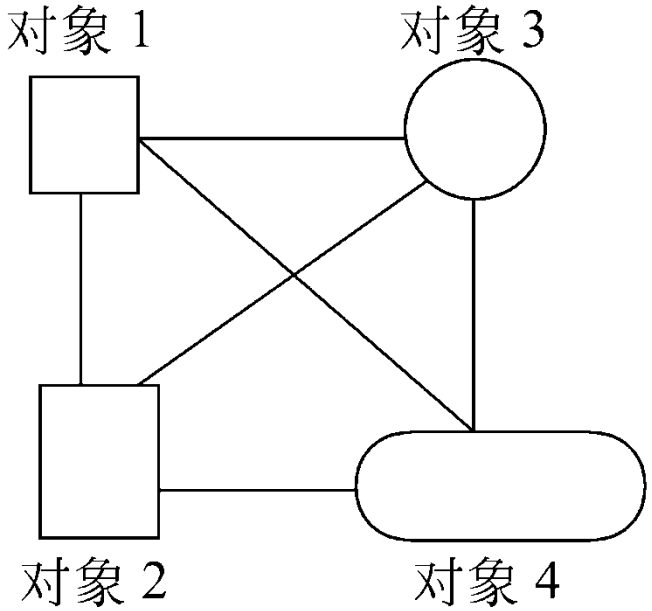
面向对象的设计方法

- 将数据及对数据的操作方法封装在一起，作为一个相互依存、不可分离的整体——对象。
- 对同类型对象抽象出其共性，形成类。
- 类通过一个简单的外部接口与外界发生关系。
- 对象与对象之间通过消息进行通讯。

1、从面向过程到面向对象

面向对象的设计方法





对象 = 算法 + 数据结构

程序 = （对象+对象+对象+...） + 消息

1、从面向过程到面向对象

面向对象的设计方法的特点

封装性

- 数据与数据处理相结合
- 有限的对外接口
- 限制外部对类对象中数据和操作的访问权限

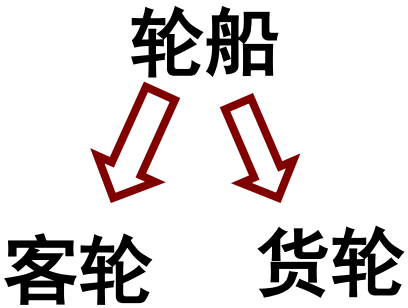
1、从面向过程到面向对象

面向对象的设计方法的特点

继承性

特殊类的对象拥有其一般类的全部属性与服务。

定义一个新的类时，新的类与已定义的类比，仅是增加或者修改了部分属性和操作，新的类可以继承原来的类。



继承是C++中支持层次分类的一种机制，允许程序员在保持原有类特性的基础上，进行更具体的说明。

1、从面向过程到面向对象

面向对象的设计方法的特点

多态性

同样的消息被不同的对象接收产生不同的结果

同一接口，多种功能方法

中学生

数学、语文、英语

计算平均成绩

大学生

高数、线代、英语

2、类与对象的概念

2、类与对象的概念

一般意义上的对象：

现实世界中一个实际存在的事物，可以是有形的（如一辆汽车），也可是无形的（如一项计划）。

对象是构成世界的一个独立单位，具有：

- 静态特征：可以用某种数据来描述
- 动态特征：对象所表现的行为或具有的功能

2、类与对象的概念

- 类是对象的抽象；
- 对象是类的具体实例（instance）

抽象是对具体对象（问题）进行概括，抽出这一类对象的公共性质并加以描述的过程。先注意问题的本质及描述，其次是实现过程或细节。对一个问题的抽象包括两方面：

数据抽象：描述某类对象的属性或状态。

代码抽象：描述某类对象的共有行为特征或具有功能。

2、类与对象的概念

手机

抽象

- 品牌型号
- 屏尺寸
- 摄像头
- 内存
- 系统
- 网络支持

实例化



品牌型号	Apple iphone16
屏幕尺寸	6.1英寸
摄像头	1200万像素
内存	256G
系统	IOS
网络支持	双卡双待



品牌型号	华为P70
屏幕尺寸	6.6英寸
摄像头	1300万像素
内存	256G
系统	HarmonyOS
网络支持	双卡双待

2、类与对象的概念

- 面向对象方法中的对象：是系统中用来描述客观事物的一个实体，它是用来构成系统的一个基本单位。
- 对象由一组属性和一组行为构成。
  - 属性：用来描述对象静态特征的**数据项**。
  - 行为：用来描述对象动态特征的**操作序列**。

2、类与对象的概念

类是具有相同属性和行为的一组对象的集合，它为属于该类的全部对象提供了统一的抽象描述，其内部包括属性和行为两个主要部分。

- 利用类可以实现数据的封装、隐藏、继承与派生
- 利用类易于编写大型复杂程序，其模块化程度比采用函数更高

函数是将逻辑上相关的语句与数据封装，用于完成特定的功能

类是逻辑上相关的函数与数据的封装，它是对所要处理的问题的描述

3、类的定义与对象声明

3.1、类的定义

举例：圆类的定义

数据属性：圆心坐标、半径

操作行为：

- 设置圆的参数
- 计算圆的周长
- 计算圆的面积
- 输出圆的参数

关键字

类名

```
class   Circle
{
   private:
        double x, y, r;
  public:
        void Set (double x0,double y0,double r0 )
              { x=x0,y=y0,r=r0;}
       double area(){ return 3.14159*r*r;}
       double perim(){return 2*3.14159*r;}
       void Print()
        {  cout<<“center:”<<x<<‘,’<<y;
           cout<< “radius:”<<r<<endl;
        }
};
```

3.1、类的定义

类定义声明形式：

```
class 类名称
{
    public:
        公有成员
    private:
        私有成员
    protected:
        保护型成员
};
```

分号不可少

公有成员：在关键字**public**后面声明，它们是类与外部的接口，任何外部函数都可以访问公有类型数据和函数。

私有成员：在关键字**private**后面声明，只允许本类中的函数访问，而类外部的函数都不能访问。

*如果紧跟在类名称的后面声明私有成员，则关键字**private**可以省略。*

保护型成员：与**private**类似，其差别表现在继承与派生时对派生类的影响不同

3.1、类的定义

类体中定义的类成员包括函数成员和数据成员，成员函数与成员数据的定义不分先后；

函数成员可在类体内定义，也可以先说明函数原型，再在类体外定义函数体。

```
class  Clock
{
  public:
     void ShowTime()
     { cout<<Hour<<":"<<Minute<<":"<<Second;}
    void SetTime(int NewH, int NewM, int NewS);
   private:
     int Hour, Minute, Second;
};
```

函数成员

函数定义在类内

在类内声明函数

数据成员

3.1、类的定义-成员函数

在类体外定义成员函数的格式：

数据类型 类名::函数名(参数表) {..... //函数体 }

域解析运算符

```
void Clock :: SetTime(int NewH, int NewM, int NewS)
{
    Hour=NewH;
    Minute=NewM;
    Second=NewS;
}
```

3.1、类的定义-成员函数

- 允许带缺省形参值的函数

```
void Clock::SetTime(int NewH=0,int NewM=0,
int NewS=0)
{
        Hour=NewH;
        Minute=NewM;
        Second=NewS;
}
```

3.1、类的定义-成员函数

允许重载函数

```
class  Test
{
        int  x ,  y;
        int  m,  n;
   public:
      void Setxy(int  a, int b){x=a;    y=b;}
      void Setxy(int a,int b,int c,int d)
         { x=a;y=b;m=c;n=d;}
      void Printxy(int x)
        {cout<< “m="<<m<<'\t'<<“n="<<n<<endl;}
      void  Printxy()
       {cout<<"x="<<x<<'\t'<<"y="<<y<<endl;}
} ;
```

3.1、类的定义-成员函数

- 为了提高运行时的效率，对于较简单的函数可以声明为内联形式。
- 内联函数体中不要有复杂结构（如循环语句和switch语句）。
- 在类中声明内联成员函数的方式：
  - 将函数体放在类的声明中——隐式
  - 使用inline关键字—显式

3.1、类的定义-成员函数

内联成员函数举例(隐式)

```
class Point
{
 public:
   void Init(int initX,int initY)
   {     X=initX;
         Y=initY;    }
   int GetX( ){return X;}
   int GetY( ){return Y;}
 private:
    int X,Y;
};
```

在类体内直接定义的函数为内联函数

3.1、类的定义-成员函数

内联成员函数举例(显式)

```
class Point
{
  public:
    void Init(int initX,int initY);
    int GetX();
    int GetY();
   private:
    int X,Y;
};
```

```
inline void Point:: Init(int initX,int initY)
{
    X=initX;
    Y=initY;
}
inline int Point::GetX()
{    return X;  }
inline int Point::GetY()
{    return Y;  }
```

类体外说明该成员函数为内联函数

3.1、类的定义-成员函数

在定义一个类时，注意如下两点：

1、类具有封装性，并且类只是定义了一种结构（样板），所以类中的任何成员数据均不能使用关键字extern，auto或register限定其存储类型。

2、在定义类时，只是定义了一种导出的数据类型，并不为类分配存储空间，所以，在定义类中的数据成员时，不能对其初始化。

```
class  Test {  int  x=5,y=6;   //是不允许的
               extern  float  x; //是不允许的
}
```

3.2、对象的声明

- 类的对象是该类的某一特定实体，即类类型的变量。

声明形式：

类名　　对象名；

例：

```
Clock   myClock;
Point   Firstpoint;
Circle  c1;
```

类名 ←

→ 对象名

3.3、类成员的访问方式

- 类中成员互访
  - 直接使用成员名
- 类外访问**public** 属性的成员

  访问方式：对象名.成员名

- 不能访问私有成员或保护成员！！
- 若要访问对象的私有的数据成员，只能通过对象的公有成员函数来获取。

3.4、对象数组、对象指针

对象数组定义举例：

```
Clock  myClock[5];
```

对象指针定义举例：

```
Clock  *myClock;
```

通过指向对象的指针访问成员

```
myClock->ShowTime()
(*myClock).ShowTime()
```

3.5、类的应用举例

类的应用举例

类的声明

```
class Cgoods
{
private :
    char    Name[21] ;
    int     Amount ;
    float   Price ;
    float   Total_value ;
public :
    void    RegisterGoods(char[],int,float) ;
    void    CountTotal(void) ;
    void    GetName(char[]) ;
    int     GetAmount(void) ;
    float   GetPrice(void) ;
    float   GetTotal_value(void) ;
};
```

```
void CGoods::RegisterGoods(char name[], int amount, float price)
{
        strcpy(Name , name) ;
       Amount=amount ;
       Price=price ;
}
void CGoods::CountTotal()
{
        Total_value = Price*Amount;
}
void CGoods::GetName(char name[])
{
        strcpy(name , Name);
}
int  CGoods::GetAmount(){ return Amount  ;}
float CGoods::GetPrice(){ return Price;}
float CGoods::GetTotal_value()
{
        return Total_value ;
}
```

函数成员定义

私有成员类外不能直接访问，可通过公有函数返回其值

类示例1

```
int main( ){
        CGoods          car ;
        char  str[21] ;
        int       number ;
        float  pr ;
        cout<<"请输入汽车型号：" ;
        cin.getline(str , 20) ;
        cout<<"请依次输入汽车数量与单价：" ;
        cin>>number>>pr ;
        car.RegisterGoods(str , number , pr) ;
        car.CountTotal() ;
        str[0]='\0' ;
        car.GetName(str) ;
       cout<<setw(20)<<str;
       cout<<setw(5)<<car.GetAmount() ;
        cout<<setw(10)<<car.GetPrice()
     cout<<setw(20)<<car.GetTotal_value()<<endl ;
     return 0;
}
```

通过公有函数设置商品信息

4、构造函数与析构函数

4、构造函数/析构函数

构造函数和析构函数是在类体中说明的两种特殊的成员函数。

构造函数是在创建对象时，使用给定的值来将对象初始化。

析构函数的功能正好相反，是在系统释放对象前，对对象做一些善后工作。

4.1构造函数


```
class Point
{
      float  x,y;
 public:
     float Sum() {   return  x+y;   }
     void Set(float  a,float  b) {    x=a;          y=b;}
     void Print()
          {  cout<<"x="<<x<<'\t'<<"y="<<y<<endl; }
};
int main()
{
        Point a1;
        a1.Print();
        a1.Set(10.0, 20.0);
        a1.Print();
         return 0;
}
```

输出什么？

利用成员函数为对象成员赋值



成员x，y私有属性
如何在声明对象时给一个初始化值？

构造函数提供了初始化对象的一种简单的方法。

4.1构造函数

```
class Point
{
    float x,y;
public:
    Point(float a,float b){ x=a;  y=b;}
    float Sum() {  return x+y;  }
    void Set(float a,float b) {   x=a;=b;}
    void Print() { cout<<"x="<<x<<'\t'<<"y="<<y<<endl; }
};
int main()
{     Point a1(2.0, 3.0),  a2(1.0,2.0);
      a1.Print();
      a2.Print();
      a2.Set(10.0, 20.0);
      a2.Print();
      return 0;
}
```

构造函数

构造函数是类的成员函数，系统约定构造函数名必须与类名相同。构造函数没有返回值

声明对象时调用构造函数初始化数据，构造函数在对象创建时由系统自动调用

利用成员函数重新为对象赋值

4.1构造函数

对构造函数，说明以下几点：

- 构造函数的函数名必须与类名相同。构造函数的主要作用是完成初始化对象的数据成员以及其它的初始化工作。
- 在定义构造函数时，不能指定函数返回值的类型，也不能指定为void类型。
- 一创建对象时，系统自动调用构造函数